

Article Title

Jiahuai Mao^{1,2}, Baosheng Li^{2,3}, Third Author^{1,2*}

^{1*}Department, Organization, Street, City, 100190, State, Country.

²Department, Organization, Street, City, 10587, State, Country.

³Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): iiiauthor@gmail.com;

Contributing authors: jhmao.sp@gmail.com; iiauthor@gmail.com;

A Supplementary Methods

A.1 Supplementary Method 1: Metadata and network model

Topology construction uses a compact descriptor for each active client C_i :

$$(i, T_i^{\text{loc}}, n_i, \mathbf{p}_i),$$

where i is the client identifier, T_i^{loc} is the estimated time required for the prescribed local training workload, n_i is the number of local samples, and $\mathbf{p}_i$ is a summary of the local data composition.

For the supervised classification experiments,

$$\mathbf{p}_i = (p_i^1, \dots, p_i^Q), \quad p_i^c = \frac{n_i^c}{n_i},$$

where Q is the number of classes and n_i^c is the number of class- c samples held by client C_i . The descriptor is used only for topology construction and does not contain raw training samples. The class histogram describes label composition but does not fully capture feature distributions or differences between local optimisation problems.

The training-time estimate T_i^{loc} measures the client-side wall-clock time required to complete the fixed local workload used in a cluster update. It does not include model transmission or waiting at the cluster leader. For an existing client, the estimate is updated from observed local training times using an exponential moving average:

$$T_i^{\text{loc}} \leftarrow \gamma_T T_i^{\text{loc}} + (1 - \gamma_T) T_{i,\text{obs}}^{\text{loc}}, \quad (1)$$

where $0 \leq \gamma_T < 1$. A newly joined client obtains its initial estimate from a short profiling run using the same local training configuration. The profiling workload is specified in the experimental settings.

The network model provides a bidirectional feasible communication graph over the active clients. An edge indicates that the corresponding clients can communicate directly when selected as leaders. After one leader has been selected from each cluster, the selected leaders induce the leader overlay. Inter-cluster mixing requires this overlay to be connected.

For a connected leader overlay, let d_k^s denote the degree of leader k at clustering round s . The nominal mixing weights are

$$w_{kj}^s = \begin{cases} \frac{1}{1 + \max(d_k^s, d_j^s)}, & j \in \mathcal{N}_k^s, j \neq k, \\ 1 - \sum_{\ell \in \mathcal{N}_k^s} w_{k\ell}^s, & j = k, \\ 0, & \text{otherwise,} \end{cases} \quad (2)$$

where $\mathcal{N}_k^s$ is the neighbour set of leader k . This construction gives a symmetric and doubly stochastic matrix on the connected overlay.

A.2 Supplementary Method 2: Deterministic topology construction

This section specifies the deterministic procedure used to construct a topology for the active-client set $\mathcal{C}^s$. Given the same descriptors, feasible communication graph and method parameters, the procedure returns the same topology.

Candidate cluster numbers and capacities.

At clustering round s , the candidate number of clusters is selected from

$$\mathcal{K}^s = \left\{ K_{\min}, \dots, \left\lfloor \frac{N^s}{n_{\text{cluster}}^{\min}} \right\rfloor \right\}, \quad N^s = |\mathcal{C}^s|, \quad (3)$$

where $n_{\text{cluster}}^{\min}$ is the minimum allowed number of clients in each cluster. Candidate values outside this range are not considered.

For a fixed K , let

$$r = N^s - K \left\lfloor \frac{N^s}{K} \right\rfloor.$$

The first r cluster indices are assigned capacity $\lceil N^s/K \rceil$, and the remaining clusters are assigned capacity $\lfloor N^s/K \rfloor$. The capacities are fixed before client assignment and satisfy Eq. (??).

Initial partition construction.

The sample-weighted class histogram of all active clients is

$$\mathbf{p}_G^s = \frac{\sum_{C_i \in \mathcal{C}^s} n_i \mathbf{p}_i}{\sum_{C_i \in \mathcal{C}^s} n_i}. \quad (4)$$

For each client, define

$$d_i = \|\mathbf{p}_i - \mathbf{p}_G^s\|_1. \quad (5)$$

Clients are ordered by decreasing d_i , then decreasing T_i^{loc} , and finally increasing client identifier.

The first K clients are assigned to different clusters. Each remaining client is assigned to an unfilled cluster that gives the smallest increase in the balanced clustering score J_{bal} in Eq. (??). When several clusters give the same increase, the smaller cluster index is used.

The waiting and distribution costs are normalised as

$$\bar{J}_T = \frac{J_T}{N^s (T_{\max}^s - T_{\min}^s)}, \quad \bar{J}_D = \frac{J_D}{2}, \quad (6)$$

where $T_{\max}^s$ and $T_{\min}^s$ are the largest and smallest training-time estimates among the active clients. The factor 2 is the largest possible L_1 distance between two probability vectors.

Partition refinement.

After the initial assignment, the method considers exchanges of two clients belonging to different clusters. An exchange is allowed only when both clusters retain their fixed capacities.

At each step, the eligible exchange giving the largest decrease in J_{bal} is accepted if the decrease exceeds $\varepsilon_{\text{swap}}$. The procedure stops when no such exchange is available or after R_{swap} accepted exchanges. When several exchanges give the same score change, the decision is made using cluster indices and client identifiers in ascending order.

The resulting partition is a deterministic feasible construction for the candidate K . It is not claimed to be the global minimiser of J_{bal} .

Connectivity-aware leader selection.

Leader selection is performed after the partition has been constructed. Clients in each cluster are ordered by increasing T_i^{loc} , followed by increasing client identifier. The method then searches for one leader per cluster such that the induced leader overlay is connected.

For a connected leader combination, let ℓ_k denote the selected leader of cluster k . The leader score is

$$J_L = \sum_{k=1}^K T_{\ell_k}^{\text{loc}}. \quad (7)$$

Among the connected combinations considered by the deterministic search, the combination with the smallest J_L is retained. When several combinations have the same

J_L , the ordered list of leader identifiers is used. The partition is discarded only when no choice of one leader per cluster forms a connected overlay.

Communication proxy and topology selection.

For topology comparison, let B denote the size of one transmitted model or model update, and let $E_L^s(K)$ denote the edge set of the selected leader overlay. The amortised communication proxy is

$$B_{\text{comm}}^s(K) = 2B(N^s - K) + \frac{2B|E_L^s(K)|}{H}. \quad (8)$$

The first term counts one model download and one model-update upload for each non-leader client in a standardised cycle where every cluster completes one local update. The second term counts the two directed transmissions associated with each leader-overlay edge and amortises them over the mixing interval H .

This proxy is used only to compare candidate topologies. It does not attempt to reproduce the exact communication generated by asynchronous execution. Actual transmitted bytes, including the one-off exchange used to initialise neighbour caches, are measured directly in the experiments.

For the feasible candidates, the proxy is normalised as

$$\bar{J}_C(K) = \frac{B_{\text{comm}}^s(K) - B_{\min}^s}{B_{\max}^s - B_{\min}^s}, \quad (9)$$

where $B_{\min}^s$ and $B_{\max}^s$ are the minimum and maximum proxy values among the feasible candidates. If all candidates have the same communication proxy, $\bar{J}_C(K)$ is set to zero.

The normalised value is substituted into Eq. (??). The feasible candidate with the smallest topology score is selected. When several candidates have the same score, the smaller K is used.

A.3 Supplementary Method 3: Dynamic topology refresh

At each topology refresh, the active-client set and feasible communication graph are updated before the partition is repaired. Departed clients are removed, newly joined clients are added after their descriptors have been collected, and persistent link changes are applied to the feasible communication graph.

The method first checks whether the previous number of clusters K^s remains feasible for the updated number of active clients. If it remains feasible, the previous partition is repaired before a full reconstruction is attempted.

Departed clients are first removed from their former clusters. The target cluster capacities are then recomputed using the same deterministic rule as in Supplementary Method 2. Newly joined clients are inserted into clusters with available capacity according to the smallest increase in J_{bal} .

If some clusters remain above capacity, clients are moved from oversized clusters to clusters below capacity. Among the moves that reduce the capacity violation, the method selects the move giving the smallest increase in J_{bal} . When several moves give

the same increase, the decision is made using cluster indices and client identifiers in ascending order. Once the target capacities have been restored, the exchange-based refinement in Supplementary Method 2 is applied.

Leaders are then selected again under the connected-overlay requirement. The repaired topology is retained only when it satisfies both the target capacities and the connected-overlay requirement.

The full topology-construction procedure is rerun when the previous K^s is no longer feasible or when repair cannot restore both the capacity and connectivity requirements. The reconstructed topology may use a different number of clusters.

If no connected leader overlay can be constructed, clusters with an available leader continue their local updates independently, while inter-cluster mixing is suspended. Clusters without an available leader pause their updates. Topology construction is attempted again at the next refresh. Once a connected overlay becomes available, neighbouring leaders exchange their current cluster models to initialise their caches before inter-cluster mixing resumes.

Model transfer after repartitioning.

When the partition changes, each new cluster is warm-started from the previous cluster models. For a new cluster V_k^{s+1} , let

$$R_k^{s+1} = V_k^{s+1} \cap \mathcal{C}^s$$

denote its retained clients.

If $R_k^{s+1} \neq \emptyset$, define

$$\beta_{kj}^{s+1} = \frac{\sum_{C_i \in V_k^{s+1} \cap V_j^s} n_i}{\sum_{C_i \in R_k^{s+1}} n_i}. \quad (10)$$

The new cluster model is initialised as

$$m_k^{s+1, \text{init}} = \sum_{j=1}^{K^s} \beta_{kj}^{s+1} m_j^{s, \text{last}}. \quad (11)$$

Thus, a previous cluster contributes in proportion to the retained data that it shares with the new cluster.

If the new cluster contains only newly joined clients, it is initialised from the mean of the previous cluster models:

$$m_k^{s+1, \text{init}} = \frac{1}{K^s} \sum_{j=1}^{K^s} m_j^{s, \text{last}}. \quad (12)$$

At the first clustering round, all clusters start from the same initial model.

A.4 Supplementary Note 1: Event-driven representation of asynchronous training

The implementation uses the cluster-local counters t_k defined in the main Methods. For analysis, completed cluster updates are ordered by a global event index

$$q = 0, 1, 2, \dots,$$

and a_q denotes the cluster that completes event q . This index records event order only and does not introduce a global clock or a synchronisation barrier.

Let x_k^q denote the latest state of cluster k immediately before event q . For the active cluster $k = a_q$, define

$$u_k^q = \mathcal{U}_k(x_k^q; \zeta_k^q),$$

where $\mathcal{U}_k$ represents the cluster-local training and aggregation in Eq. (??), and ζ_k^q collects the randomness of local optimisation and client participation.

If event q does not trigger inter-cluster mixing, the active cluster updates as

$$x_k^{q+1} = u_k^q.$$

If event q triggers mixing, let $\psi_{kj}(q) \leq q$ denote the event at which the cached state of neighbour j used by cluster k was generated. The update is then

$$x_k^{q+1} = w_{kk}^s u_k^q + \sum_{j \in \mathcal{N}_k^s} w_{kj}^s x_j^{\psi_{kj}(q)}. \quad (13)$$

All inactive cluster states remain unchanged.

The event-level age of the cached state is

$$\tau_{kj}(q) = q - \psi_{kj}(q). \quad (14)$$

At the start of each fixed-topology interval, neighbouring leaders exchange their current states so that every overlay edge has an initial cached model. During the interval, the overlay remains fixed and connected, every active cluster continues to obtain update opportunities, and transmitted models are eventually delivered. Each model carries a monotonically increasing sender-side counter, and a cache is updated only when a model with a larger counter is received. A delayed older message therefore cannot overwrite a newer cached state.

If a theoretical result requires bounded delay, the corresponding analysis additionally assumes

$$\tau_{kj}(q) \leq \tau_{\max}.$$

The matrix W^s defines the nominal weights used at each mixing event, but the realised process is not equivalent to synchronous multiplication by W^s . Different cluster activation frequencies and delayed cached states may change the influence of individual clusters over a finite interval. The convergence objective must therefore be established under the stated activation and delay assumptions rather than inferred from the nominal mixing matrix alone.

References